



COMP50011 - MOBILE APP DEVELOPMENT II

INDIVIDUAL ASSIGNMENT

8th of June 2026

PayPatch Mobile Application Technical Report

Lecturer: Mohanraj Kavinkumar

Submitted by:

Chanupa Dulnuwan-CB016173

COM24A1

Contents

01. Introduction.....	3
02. Test Cases	3
2.1. Authentication and Session.....	3
2.2 Navigation and Screen Flow	4
2.3 Group Management	4
2.4 Expenses, Balances, and Settlements.....	5
2.5 Friends, Contacts, and Social Feed	6
2.6 Posts, Likes, and Comments	6
2.7 Activity and Announcements.....	7
2.8 Settings, Security, and Device Features.....	7
2.9 Data Integration, Offline Support, and Local Storage	8
2.10. Validation and Robustness.....	9
03. System Design and Technical Implementation.....	9
3.1 Project File Structure and MVC Architecture	10
3.2 Frontend Mobile Application.....	10
3.3 State Management.....	11
3.4 Backend API and Database.....	11
3.5 Authentication and Session Handling.....	12
3.6 Main Functional Features	12
3.7 Offline Support and Local Storage	12
3.8 Device Features Used	13
04. Application User Interface Screenshots.....	13
05. LinkedIn Learning Course	15
06. Summary and Conclusion	15

01. Introduction

This report presents the PayPatch mobile application, which was developed to manage shared expenses, groups, friends, settlements, and user activity in a simple way. The report includes the main technical details of the system, test cases used to check the application, screenshots of the user interface, and a short reflection on the LinkedIn Learning course completed for this project.

02. Test Cases

2.1. Authentication and Session

TC ID	Test Case	Preconditions / Data	Expected Result	Actual Result	Status
AUTH-01	Login with valid email and password	Registered user exists and API is reachable	User is authenticated and redirected from Login to Home screen	User was authenticated and redirected from Login to Home screen successfully	Pass
AUTH-02	Login with invalid password	Registered email exists with incorrect password entered	Error message is shown and user stays on Login screen	Error message was shown and the user remained on Login screen	Pass
AUTH-03	Register a new account with valid data	Name, email, password, confirm password, and country provided	New account is created and the user is signed in	New account was created and the user was signed in successfully	Pass
AUTH-04	Register with mismatched passwords	Password and confirm password do not match	Validation alert is displayed and registration does not continue	Validation alert was displayed and registration did not continue	Pass
AUTH-05	Register with invalid optional username format	Username contains unsupported characters	Validation alert is displayed for the username format	Validation alert was displayed for the username format	Pass
AUTH-06	Continue with Google from Login	Google demo profile flow is available	User is logged in with the demo Google account and enters the app	User was logged in with the demo Google account and entered the app successfully	Pass
AUTH-07	Continue with Google from Register	Register screen is open	User is logged in with the demo Google account and enters the app	User was logged in with the demo Google account and entered the app successfully	Pass
AUTH-08	Persist session after app restart	User has already logged in once	Saved token and user data are restored and Splash routes to Home	Saved token and user data were restored and Splash routed to Home correctly	Pass
AUTH-09	Logout from profile sheet	User is authenticated	Token and saved session are cleared and app returns to Login	Token and saved session were cleared and the app returned to Login	Pass
AUTH-10	Configure Laravel API base URL from Login	User opens the API configuration dialog	New API base URL is saved locally and reused on the next app launch	New API base URL was saved locally and reused on the next app launch	Pass

2.2 Navigation and Screen Flow

TC ID	Test Case	Preconditions / Data	Expected Result	Actual Result	Status
NAV-01	Splash screen routing for authenticated user	Valid saved session exists	Splash screen routes to Home screen automatically	Splash screen routed to Home screen automatically	Pass
NAV-02	Splash screen routing for guest user	No saved session exists	Splash screen routes to Login screen automatically	Splash screen routed to Login screen automatically	Pass
NAV-03	Bottom navigation between the 4 main tabs	User is logged in	User can switch between Groups, Friends, Activity, and Settings screens	User switched between Groups, Friends, Activity, and Settings screens successfully	Pass
NAV-04	Open Register screen from Login	Login screen is visible	Register screen opens from the sign-up action	Register screen opened from the sign-up action successfully	Pass
NAV-05	Open Group Detail from Groups list	At least one group exists	Selected group opens in the detailed group screen	Selected group opened in the detailed group screen successfully	Pass
NAV-06	Open secondary settings pages	Settings screen is visible	Insights, Security, Announcements, and Announcement Detail screens open correctly	Insights, Security, Announcements, and Announcement Detail screens opened correctly	Pass

2.3 Group Management

TC ID	Test Case	Preconditions / Data	Expected Result	Actual Result	Status
GRP-01	View groups list from backend	User is authenticated and online	Groups are fetched from the API and displayed in the list/grid	Groups were fetched from the API and displayed in the list/grid correctly	Pass
GRP-02	Create a new group with name and currency	User is online and enters a valid group name	New group is created and appears in the Groups screen	New group was created and appeared in the Groups screen successfully	Pass
GRP-03	Create a group with selected members	User searches and selects members before creation	Selected members are attached to the new group	Selected members were attached to the new group successfully	Pass
GRP-04	Open group detail header actions	Group detail screen is open	Header shows group name, member action, edit action for editors, and refresh action	Header showed group name, member action, edit action for editors, and refresh action correctly	Pass
GRP-05	Edit group name and currency	Group owner/editor opens Edit Group sheet	Updated group name and currency are saved and reflected in the header card	Updated group name and currency were saved and reflected in the header card correctly	Pass
GRP-06	Change group cover image	Group owner/editor selects a new cover image	Group cover image is updated in the edit view and header card	Group cover image was updated in the edit view and header card successfully	Pass
GRP-07	Change group profile image	Group owner/editor selects a new profile image	Group profile image is updated in the edit view and header avatar	Group profile image was updated in the edit view and header avatar successfully	Pass
GRP-08	Add a member to an existing group	Group owner/editor searches for a user in Members sheet	New member is added and member count increases	New member was added and member count increased correctly	Pass

GRP-09	Remove a member from an existing group	Group owner/editor removes a non-owner member	Selected member is removed from the group	Selected member was removed from the group successfully	Pass
GRP-10	Delete a group	Group owner/editor confirms delete action	Group is deleted and removed from the Groups screen	Group was deleted and removed from the Groups screen successfully	Pass
GRP-11	Leave group as a non-owner member	Current user is not the group owner	User leaves the group after confirmation and the group disappears from the list	User left the group after confirmation and the group disappeared from the list	Pass
GRP-12	Pull-to-refresh on group detail	Group detail screen is open and online	Group details are reloaded from the API	Group details were reloaded from the API successfully	Pass

2.4 Expenses, Balances, and Settlements

TC ID	Test Case	Preconditions / Data	Expected Result	Actual Result	Status
EXP-01	Add a new expense with title, amount, and payer	Group has at least one member and user is online	Expense is created and immediately appears in the Expenses list	Expense was created and immediately appeared in the Expenses list	Pass
EXP-02	Add an expense with custom split members	Group has multiple members and split members are selected	Expense is saved with the selected split members	Expense was saved with the selected split members successfully	Pass
EXP-03	Attach receipt photo when adding an expense	Camera permission is available	Receipt image is attached to the expense and stored successfully	Receipt image was attached to the expense and stored successfully	Pass
EXP-04	Capture geolocation while adding an expense	Location permission is available	Expense location metadata is captured and submitted with the expense	Expense location metadata was captured and submitted with the expense successfully	Pass
EXP-05	Open receipt image from an expense card	Expense has a receipt image URL	Receipt opens in a preview dialog	Receipt opened in a preview dialog successfully	Pass
EXP-06	Delete own expense	Current user created the expense	Delete confirmation appears and the expense is removed after confirmation	Delete confirmation appeared and the expense was removed after confirmation	Pass
EXP-07	Restrict delete action for non-creator	Current user did not create the expense	Delete option is not shown for the expense	Delete option was not shown for the expense	Pass
EXP-08	Show correct summary balance in group card	Group has expenses and balances available	Group summary shows owed/owe state with correct currency formatting	Group summary showed owed/owe state with correct currency formatting	Pass
EXP-09	Show USD to LKR approximate conversion	Group currency is USD and exchange rate is available	Approximate LKR value is shown in the group summary and cards	Approximate LKR value was shown in the group summary and cards correctly	Pass
EXP-10	Open balances sheet	Group detail screen is open	Balances sheet opens and displays member balance breakdown	Balances sheet opened and displayed the member balance breakdown correctly	Pass
EXP-11	Record settlement between members	Settlement sheet is opened with valid payer/receiver and amount	Settlement is recorded and balances refresh immediately	Settlement was recorded and balances refreshed immediately	Pass
EXP-12	Send reminders to members with pending balances	Group has members with outstanding balances	Reminder request is sent successfully for selected members	Reminder request was sent successfully for selected members	Pass

2.5 Friends, Contacts, and Social Feed

TC ID	Test Case	Preconditions / Data	Expected Result	Actual Result	Status
FRD-01	Load friends balances from backend	User is authenticated and online	Friends list loads with balance status and currency	Friends list loaded with balance status and currency correctly	Pass
FRD-02	Separate active and settled friends	Friends data contains both outstanding and settled balances	Friends screen shows Active and Settled sections correctly	Friends screen showed Active and Settled sections correctly	Pass
FRD-03	Open Add Friends invite sheet	User taps add-friend FAB while online	Invite sheet opens with tabs for adding friends	Invite sheet opened with tabs for adding friends successfully	Pass
FRD-04	Request phone contacts permission	Contacts tab is opened	Contacts permission request is shown and handled correctly	Contacts permission request was shown and handled correctly	Pass
FRD-05	Match phone contacts with PayPatch users	Contacts are available and API is reachable	Matching contacts are highlighted as existing PayPatch users	Matching contacts were highlighted as existing PayPatch users correctly	Pass
FRD-06	Show contacts permission denied state	User denies contacts permission	Clear permission denied message is displayed	Clear permission denied message was displayed	Pass
FRD-07	Display friends feed stories	Posts exist for group or friends audience	Story bubbles appear at the top of the Friends screen	Story bubbles appeared at the top of the Friends screen successfully	Pass
FRD-08	Open story viewer from feed bubble	At least one story bubble is visible	Story viewer opens and shows the selected user's posts	Story viewer opened and showed the selected user's posts correctly	Pass

2.6 Posts, Likes, and Comments

TC ID	Test Case	Preconditions / Data	Expected Result	Actual Result	Status
PST-01	Open Create Post from group detail	User is on a group they can interact with	Create Post screen opens from the Share Post FAB	Create Post screen opened from the Share Post FAB successfully	Pass
PST-02	Create post with caption only	User enters caption and valid audience	Post is created successfully and appears in the feed	Post was created successfully and appeared in the feed	Pass
PST-03	Create post with image from camera or gallery	User selects an image and valid audience	Post with image uploads successfully and appears in the feed	Post with image uploaded successfully and appeared in the feed	Pass
PST-04	Switch audience between Group and Friends	Create Post screen is open	Selected audience is applied to the post request	Selected audience was applied to the post request correctly	Pass
PST-05	Like a post from the story viewer	Story viewer is open on a post	Like count and liked state update successfully	Like count and liked state updated successfully	Pass
PST-06	Add comment to a post	Story viewer comment sheet is open and comment text is entered	Comment is submitted and appears in the comments list	Comment was submitted and appeared in the comments list successfully	Pass
PST-07	Open comments list for a post	Story viewer is open	Existing comments are fetched and displayed in the comments sheet	Existing comments were fetched and displayed in the comments sheet correctly	Pass
PST-08	Delete own post	Current user owns the selected post	Post is deleted and removed from the feed/viewer	Post was deleted and removed from the feed/viewer successfully	Pass

2.7 Activity and Announcements

TC ID	Test Case	Preconditions / Data	Expected Result	Actual Result	Status
CT-01	Load activity screen with expense history	User has groups and expenses	Activity screen shows expense history items from group data	Activity screen showed expense history items from group data correctly	Pass
CT-02	Load backend activity logs	User is online and authenticated	Activity screen shows reminders, settlements, likes, and comment logs	Activity screen showed reminders, settlements, likes, and comment logs correctly	Pass
CT-03	Clear activity badge after refresh	Unread activity count exists	Refresh clears the unread badge count	Refresh cleared the unread badge count successfully	Pass
CT-04	Open group from tappable activity item	Reminder or settlement log references a valid group	Tapping the log navigates to the related group detail screen	Tapping the log navigated to the related group detail screen successfully	Pass
NN-01	Load announcements from external JSON when online	Internet connection is available	Announcements list loads from the external JSON source	Announcements list loaded from the external JSON source successfully	Pass
NN-02	Load announcements from local asset when offline	Internet connection is unavailable	Announcements list loads from local JSON fallback	Announcements list loaded from the local JSON fallback successfully	Pass
NN-03	Open announcement detail	Announcements list contains at least one item	Detail page shows title, content, date, author, and priority	Detail page showed title, content, date, author, and priority correctly	Pass

2.8 Settings, Security, and Device Features

TC ID	Test Case	Preconditions / Data	Expected Result	Actual Result	Status
SET-01	Toggle dark theme from Settings	Settings screen is open	Theme changes between light and dark mode	Theme changed between light and dark mode successfully	Pass
SET-02	Persist selected theme after app restart	User changed the theme previously	Selected theme is restored on next app launch	Selected theme was restored on the next app launch correctly	Pass
SET-03	View battery status card	Device supports battery status plugin	Battery level and battery state are displayed correctly	Battery level and battery state were displayed correctly	Pass
SET-04	Refresh battery reading	Settings battery card is visible	Latest battery level is fetched when refresh is tapped	Latest battery level was fetched when refresh was tapped	Pass
SET-05	Open Insights screen	Settings screen is open	Insights screen opens and shows summaries/charts or empty state	Insights screen opened and showed summaries/charts or the correct empty state	Pass
SET-06	Open Security screen	Settings screen is open	Security screen opens with password and account options	Security screen opened with password and account options correctly	Pass
SET-07	Change password with valid inputs	Current password and new password are entered correctly	Password is updated and success feedback is shown	Password was updated and success feedback was shown	Pass

SET-08	Change password with wrong current password	Incorrect current password is entered	Error feedback is shown and password is not changed	Error feedback was shown and the password was not changed	Pass
SET-09	Update profile image from profile sheet	User picks camera or gallery image	Profile photo is uploaded and refreshed in the UI	Profile photo was uploaded and refreshed in the UI successfully	Pass
SET-10	Show delete-account support alert	User taps Delete Account from Security	Informational alert is shown instead of silent failure	Informational alert was shown instead of silent failure	Pass

2.9 Data Integration, Offline Support, and Local Storage

TC ID	Test Case	Preconditions / Data	Expected Result	Actual Result	Status
DAT-01	Cache groups list for offline use	User has fetched groups while online	Groups response is cached locally for later offline viewing	Groups response was cached locally for later offline viewing	Pass
DAT-02	View groups while offline	Cached group list exists and device is offline	Cached groups are shown instead of a blank screen	Cached groups were shown instead of a blank screen	Pass
DAT-03	Cache group details for offline use	User has opened a group while online	Group detail response is cached locally	Group detail response was cached locally successfully	Pass
DAT-04	View group details while offline	Cached group detail exists and device is offline	Cached group detail is displayed with safe fallback values if needed	Cached group detail was displayed with safe fallback values correctly	Pass
DAT-05	View friends while offline	Friends were fetched previously while online	Cached friends data is shown when offline	Cached friends data was shown when offline	Pass
DAT-06	Fallback to local sample groups when no cache exists	Device is offline and no cached group data exists	Local sample group data is displayed as fallback	Local sample group data was displayed as fallback successfully	Pass
DAT-07	Persist session token and user profile locally	User logs in successfully	Token and user object are saved with SharedPreferences	Token and user object were saved with SharedPreferences successfully	Pass
DAT-08	Persist API base URL locally	User changes server URL from Login screen	Base URL is saved and reused across app launches	Base URL was saved and reused across app launches correctly	Pass

2.10. Validation and Robustness

TC ID	Test Case	Preconditions / Data	Expected Result	Actual Result	Status
VAL-01	Prevent expense creation with empty required fields	Expense dialog is open and required fields are blank	Validation alert is shown and expense is not submitted	Validation alert was shown and the expense was not submitted	Pass
VAL-02	Prevent group creation with empty group name	Create Group dialog is open with blank name	Validation alert is shown and group is not created	Validation alert was shown and the group was not created	Pass
VAL-03	Prevent editing group with empty name	Edit Group sheet is open with blank name	Validation alert is shown and update is not submitted	Validation alert was shown and the update was not submitted	Pass
VAL-04	Show offline restriction for actions that require internet	Device is offline and user tries to create groups or add expenses	User is informed that the action requires an online connection	User was informed that the action requires an online connection	Pass
VAL-05	Show empty states for lists with no data	Relevant module has no data to display	Friendly empty-state message is shown instead of broken UI	Friendly empty-state message was shown instead of broken UI	Pass
VAL-06	Keep app stable during API or timeout issues	Server returns error or takes too long	User receives readable feedback and app does not crash	User received readable feedback and the app did not crash	Pass

03. System Design and Technical Implementation

This section explains the main technical details of my PayPatch application in a simple way. PayPatch was developed as a mobile application for managing shared expenses, groups, friends, settlements, posts, and activity updates. The main purpose of the system is to make group expense sharing easier, especially when users need to record expenses, split amounts, check balances, and settle payments between members.

The application was built using Flutter for the mobile frontend and Laravel for the backend API. I used Flutter because it allowed me to build a mobile application with one codebase, while still creating a clean and modern user interface. The backend was developed using Laravel because it provides a structured way to handle users, groups, expenses, authentication, and database operations. The data is stored in a MySQL database, and the mobile application communicates with the backend through API requests.

3.1 Project File Structure and MVC Architecture

The PayPatch project was organized with a clear file structure in both the Flutter frontend and Laravel backend. This helped me keep the code clean and easier to manage. In the Flutter project, the main files are inside the lib folder. Files such as main.dart and config.dart handle the app starting point and basic configuration. The models folder includes files like group.dart and activity_item.dart, which are used to represent data received from the backend. The providers folder includes files such as auth_provider.dart and groups_provider.dart, which handle app logic, API requests, and state updates. The screens folder contains the user interface pages such as Login, Groups, Friends, Activity, Settings, and Group Detail. Reusable design elements are kept inside the widgets and theme folders.

The system follows the **MVC architecture**, which means **Model**, **View**, and **Controller**. In PayPatch, the **Model** part is used to handle data. This includes Laravel database models such as Group and Expense, as well as Flutter model classes that convert API data into Dart objects. The **View** part is the user interface, mainly the Flutter screens and widgets that users interact with. Examples include the Login screen, Groups screen, Friends screen, and Settings screen. The **Controller** part manages the logic between the data and the interface. In Flutter, the provider files act like controllers because they send API requests and update the UI. In Laravel, ApiController and PostController handle backend logic, validation, and database updates.

For example, when a user creates a new group, the Groups screen works as the **View**. The GroupsProvider works as the **Controller** by sending the request to the backend. Then the Laravel controller receives the request and uses the Group **Model** to save the group in the database. After that, the response is sent back to Flutter, and the screen updates to show the new group.

3.2 Frontend Mobile Application

The frontend of PayPatch was developed using Flutter and Dart. Flutter was useful because it helped me create different screens such as Login, Register, Groups, Friends, Activity, Settings, Group Details, Create Post, Security, Insights, and Announcements. These screens were designed to give the user a simple flow through the application.

The application uses bottom navigation for the main screens. The four main sections are Groups, Friends, Activity, and Settings. This made the app easier to use because the user can quickly move between the most important parts of the system. For example, the Groups screen is used to view and manage expense groups, while the Friends screen shows balances with friends. The Activity screen displays recent actions such as settlements, reminders, likes, and comments. The Settings screen includes profile, theme, security, and device-related features.

I also used reusable widgets and a common theme structure to keep the design consistent. For example, the app supports light and dark themes, and the selected theme is saved so it can be restored when the app opens again. This improved the user experience because the app does not reset the user's preferences every time.

3.3 State Management

For state management, I used Provider. In simple terms, Provider helped the app share data between screens without passing everything manually from one screen to another. For example, after the user logs in, the authentication token and user details need to be available for other parts of the app. Provider made this easier by keeping important data in one place and updating the interface when the data changed.

The main logic of the app is handled through provider classes. For example, the authentication provider manages login, registration, logout, and session saving. The group-related provider manages group loading, group creation, group details, expenses, balances, and settlements. This structure helped me separate the user interface from the main app logic. Because of that, the code became easier to understand and maintain.

3.4 Backend API and Database

The backend of PayPatch was developed using Laravel. The Laravel backend works as the central part of the system because it receives requests from the mobile app, processes them, and sends responses back as data. For example, when a user logs in, creates a group, adds an expense, or records a settlement, the mobile app sends a request to the Laravel API.

The backend uses models to represent main data objects such as users, groups, expenses, members, posts, comments, and settlements. These models connect with the MySQL database. This helped the system store and retrieve data in an organized way. Laravel also helped with validation, so incorrect or incomplete data does not get stored in the database.

The system follows a simple Model-View-Controller idea. In my app, the Flutter screens act as the view because they show the interface to the user. The provider classes and Laravel controller logic act as the control part because they handle actions and decisions. The database models act as the model part because they represent the data. This separation helped the project become more organized instead of mixing all the code in one place.

3.5 Authentication and Session Handling

Authentication is an important part of PayPatch because users need to access their own groups, expenses, friends, and activity details. The app supports login, registration, Google demo login, session saving, and logout. After a successful login, the user's token and profile details are saved locally. This allows the user to stay logged in even after closing and reopening the app.

When the app starts, it checks whether a saved session is available. If a valid session exists, the user is taken to the home screen. If not, the user is taken to the login screen. This made the app feel more practical because the user does not need to log in every time.

For security, logout clears the saved token and user data. After logout, the user is returned to the login screen. This is important because the user's personal expense and group data should not remain accessible after logging out.

3.6 Main Functional Features

The main feature of PayPatch is group expense management. A user can create a group, add members, edit group details, change group images, add expenses, attach receipt images, and view group balances. The system also allows users to record settlements between members. This is useful because it helps users see who owes money and who should receive money.

The expense feature includes important details such as title, amount, payer, selected split members, receipt image, and location data. The receipt image feature makes the expense more trustworthy because users can keep proof of payment. The location feature adds more context to the expense, especially when recording where the transaction happened.

The app also includes social features such as posts, likes, comments, and story viewing. These features make the application more interactive, not only a basic calculator for expenses. Users can create posts for groups or friends, view stories, like posts, and comment. This adds a social side to the expense-sharing system.

3.7 Offline Support and Local Storage

One important part of the application is offline support. Since users may not always have an internet connection, the app saves some data locally. For example, group data, group details, friends data, session data, and API configuration can be saved on the device. If the user is offline, the app can show cached data instead of showing a blank screen.

This feature improved the reliability of the app. Even when the server is not reachable or the internet connection is not available, the user can still view previously loaded data. However, actions such as creating groups or adding expenses still need an internet connection because they must be saved in the backend database.

The app also uses local storage for preferences such as the selected theme and API base URL. This helped the app remember user settings and made testing easier because the API URL can be changed from the login screen.

3.8 Device Features Used

PayPatch also uses some mobile device features. The camera or gallery can be used to upload profile images, group images, post images, and receipt images. The location feature can be used when adding expenses. The contacts feature helps users find possible friends from their phone contacts. The battery status feature is also shown in the Settings screen.

These features helped the app feel more like a real mobile application instead of only a simple form-based project. At the same time, I had to think about permissions because features like camera, location, and contacts need user permission before they can be used.

04. Application User Interface Screenshots

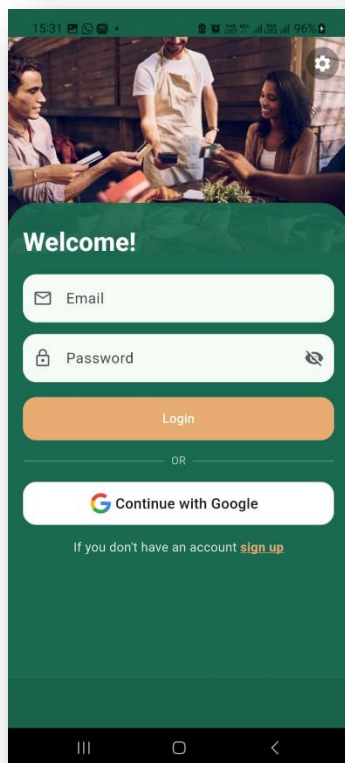


Figure 3: Logging Page

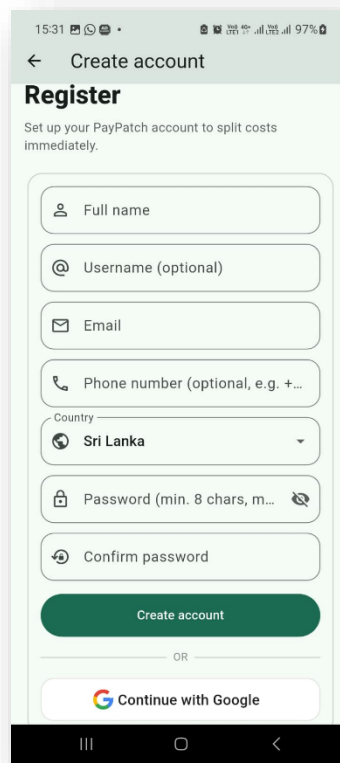


Figure 2: Register Page

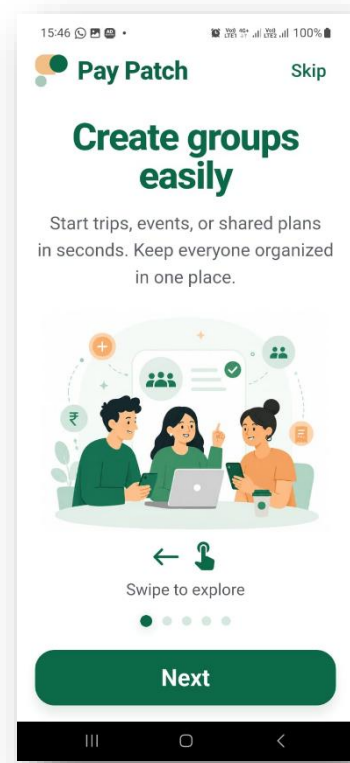


Figure 1: Into Slides

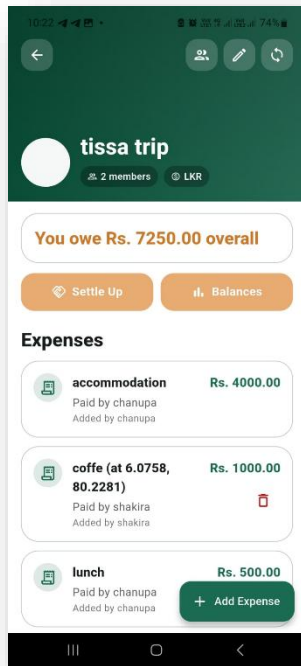


Figure 9: Group Details Page

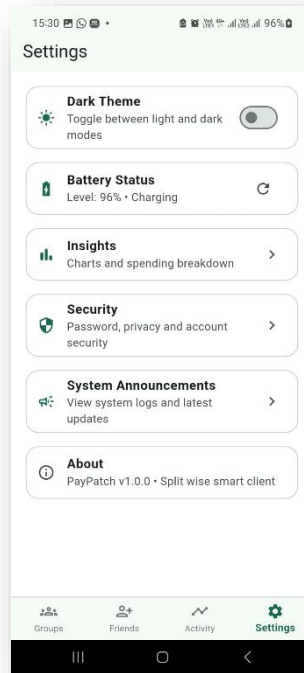


Figure 6: Settings Page

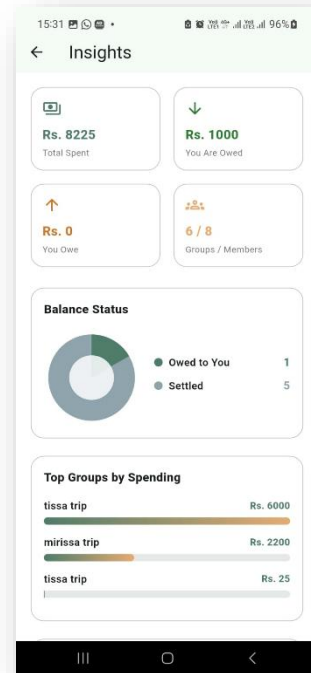


Figure 5: Insights Page

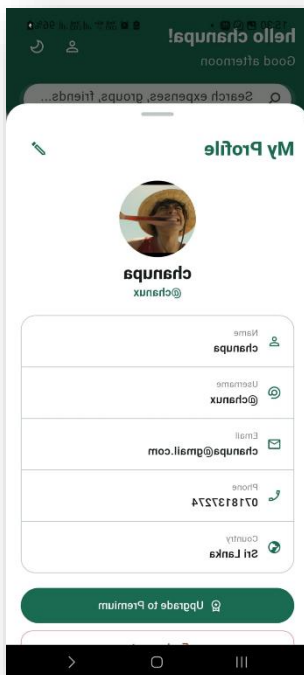


Figure 4: Profile page

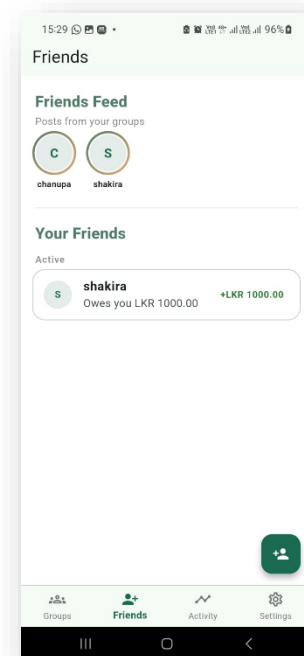


Figure 8: Friends Page

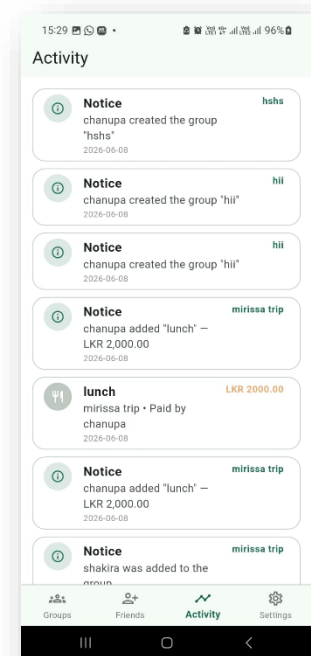


Figure 7: Activity page

05. LinkedIn Learning Course



Figure 10: Certificate Of Course

06. Summary and Conclusion

In conclusion, this report presented the PayPatch mobile application, including its test cases, system architecture and technical implementation, application screenshots, and LinkedIn Learning course reflection. Through this project, I was able to understand how a mobile frontend, backend API, database, and testing process work together in a complete application. Overall, PayPatch helped me improve my practical development knowledge and understand the importance of planning, structure, and testing in software development.

The GitHub repository link and LinkedIn Learning certificate link are provided below.

[Git Repo Link](#)

[LinkedIn Learning certificate](#)